Holistic Workload Quantification of Synchronously Invoked APIs for Near-One-Step Microservice Autoscaling

Anonymous Author(s)

Abstract

The essence of autoscaling for microservice systems is to map diverse runtime metrics—primarily latency, requests per second (RPS), and CPU usage—into the resources required for each service to match the current workload from users. This implies that a metric should always form a one-to-one mapping with the current workload size to serve as a qualified autoscaling metric; otherwise, if the workload size cannot be uniquely determined based on the metric, the required resources to match the workload cannot be determined either. Most existing studies are conducted on the assumption that latency, RPS, and CPU usage can fully quantify the workload. However, our empirical analysis reveals that for APIs invoked synchronously, these metrics fail to provide an accurate quantification. Consequently, autoscalers relying on these metrics often struggle to perceive the actual workload size, leading to inefficient trial-and-error cycles when searching for optimal resource allocation. This keeps the system in a prolonged state of resource waste or SLO violations.

In this work, we move beyond traditional metrics and propose a novel method for quantifying the workload of synchronously invoked APIs with high accuracy and robustness. This method significantly simplifies the autoscaling problem, enabling us to develop Squanler, a workload-based autoscaler that can maintain end-to-end latency at any feasible service level objective (SLO) with near-minimal resource redundancy. Comparative experiments across two microservice benchmarks show that Squanler achieves the lowest SLO violation rates while saves 39%–53% of the resources required by state-of-the-art autoscalers, and additional evaluations demonstrate its superior accuracy and robustness.

CCS Concepts

• **Computer systems organization** → **Cloud computing**; • **Software and its engineering** → **Software performance**.

Keywords

Microservices Autoscaling, Synchronous Invocation, Workload Quantification, QoS Guarantee, Resource Efficiency

ACM Reference Format:

Anonymous Author(s). 2018. Holistic Workload Quantification of Synchronously Invoked APIs for Near-One-Step Microservice Autoscaling. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX).* ACM, New York, NY, USA, 11 pages. https://doi.org/XXXXXXX.XXXXXXX

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

The microservice architecture is widely adopted for online systems and has become the de facto standard in cloud computing [7, 13, 14, 28]. Unlike traditional monolithic architectures, this design paradigm decomposes applications into small, independently deployable services, each built around specific business capabilities and collaborating through lightweight communication protocols [15, 18, 27, 36]. Microservice applications generate a large volume of metrics that contain essential information about system runtime. Among these metrics, latency, requests per second (RPS), and CPU usage are widely adopted by existing autoscalers [3, 8, 10, 16, 23, 24, 33, 34] to map the required resources of each service: the resource that achieves the lowest SLO violation rate and the least resource redundancy. In other words, it is the resources that perfectly match the current workload.

From an information-theoretic perspective, the workload contains the necessary information to infer the required resources. Without knowing the specific size of the current workload, it is impossible to calculate the resources needed to match it. However, after examining the bijective relationship between latency, RPS, CPU usage, and workload, we find that these three traditional metrics often poorly reflect workload size (Sec. 2). In some cases, they show identical values under different workloads, while in others they take different values under the same workload. As a result, we cannot infer the exact workload from them, and thus cannot derive the information necessary to determine the required resources. Building autoscalers based on these metrics is therefore an inadequate starting point. Such autoscalers lack an exact perception of the workload size and become trapped in a trial-and-error process, forced to iteratively solve the problem using fragmented information, which incurs high operational costs for cluster maintenance.

The exact size of the workload is necessary for autoscaling. Therefore, how can workload be accurately quantified? Traditional studies typically use RPS to quantify the workload an API faces. RPS = 100 means users are sending 100 requests per second. Consequently, traditional autoscalers calculate the required resources to handle these 100 requests [1, 8, 23, 33] or use RPS as an input feature for their models [8, 23, 30]. However, we find that in synchronously invoked APIs, RPS fails to quantify the workload (More accurately, latency, RPS, and CPU usage all fail to do that): when users invoke an API synchronously, they send a request and must wait for a response. Only after receiving the response can the user send the next request. This constitutes the lifecycle of a request (Fig. 1). We define the total time a request spends inside the cluster gateway (typically related to computation and queuing) as inside latency, and the total time spent outside the gateway (typically related to network RTT and user think time) as outside latency. In

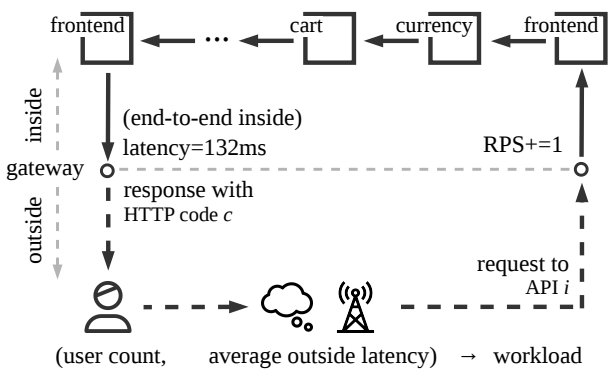


Figure 1: Request lifecycle in synchronously invoked APIs and our novel way to redefine a workload.

this process, higher inside and outside latencies both delay the response, forcing users to wait longer and issue the next request later. Latency, acting as backpressure, slows down the user's sending rate and thus throttles the RPS in turn, making RPS no longer an independent variable. The observed RPS = 100 is merely the result of throttling, while the actual RPS users want to send could be much higher. This makes RPS an inappropriate metric to quantify the workload for synchronously invoked APIs anymore.

We propose a more fundamental workload quantification method based on the request initiator—the user. When using load testing tools [2, 17, 29] to synchronously inject a workload into an API, we essentially simulate a group of users who issue requests and wait for responses continuously. In this context, the user count and the average outside latency (denoted as AOL, which in workload configuration files typically manifests as the average think time between a user receiving a response and issuing the next request) are two key configurable parameters sufficient to define a workload. Therefore, we can use "the user count under a specified AOL" to quantify a workload from its root. When we say "workload = 150 (1s)," it means the current workload is equivalent to that generated by 150 users with a 1s AOL. This new quantification ensures that the workload can be standardized and reproduced by its fundamental definition, rather than by its observed effects.

In Sec. 3, we introduce the equation to obtain this new quantification with high precision and robustness. Leveraging this new quantification, we develop Squanler, a simple yet efficient workload-based autoscaler that calculates the required resources to exactly match the workload in nearly one step (Sec. 4). Experiments on two open-source microservice benchmarks show that Squanler demonstrates superior accuracy and robustness, achieving the lowest SLO violation rates while allocating roughly half the resources required by state-of-the-art baselines (Sec. 5). The data and code are provided in Sec. 9.

Our key contributions are as follows.

- **An empirical analysis of information incompleteness in traditional metrics.** We demonstrate that latency, RPS, and CPU usage lack the full information to infer the workload of synchronously invoked APIs. This reveals the insurmountable performance ceiling shared by existing autoscalers. We are the first to identify and comprehensively analyze this incompleteness in the context of autoscaling.
- **A more appropriate quantification of workload.** We introduce "the user count under a specified AOL" as a more appropriate quantification for the workload on synchronously invoked APIs and derive an equation to obtain it. This enables the direct use of workload as a simple, accurate, and robust metric for autoscaling.
- **The application of workload in autoscaling.** We develop Squanler, a workload-based autoscaler for microservices. Evaluations on two open-source benchmarks show that Squanler significantly outperforms state-of-the-art autoscalers in both SLO maintenance and resource efficiency, demonstrating the practical value of this new workload quantification.

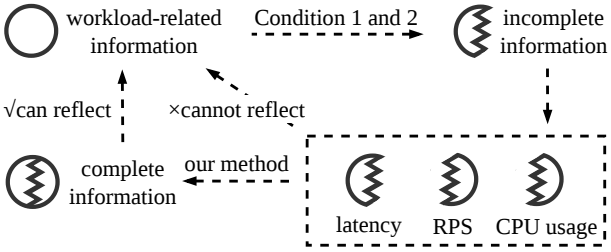


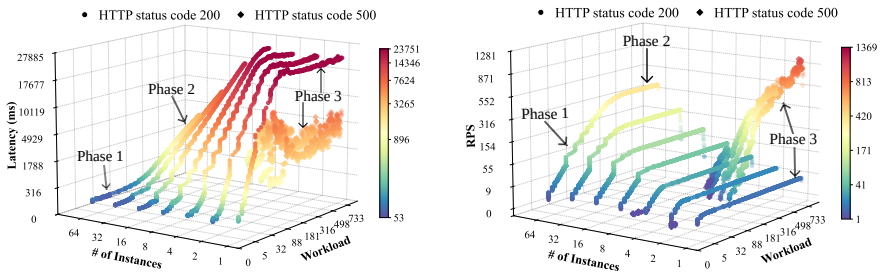
Figure 2: The information loss in traditional metrics and the potential for information reintegration.

2 Background and Motivation

Information completeness is the focus of our discussion. Therefore, we simplify the workload-related information within metrics as "information". In this section, we analyze the poor bijectivity between latency, RPS, CPU usage, and workload in synchronously invoked APIs to prove that the information they provide is severely insufficient for a qualified autoscaling metric. We summarize this into two conditions. Condition 1: latency, RPS, and CPU usage can take the same values for different workloads. Thus, they naturally convey fragmented information. Condition 2: latency, RPS, and CPU usage can take different values for the same workload. This occurs because they are co-determined by the workload and other external noise, such as scaling actions, server crashes, or resource contention. Such noise causes the metrics to fluctuate even under a constant workload, further distorting the already fragmented information. The entire process is shown in Fig. 2.

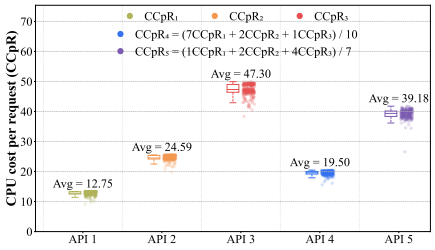
2.1 Condition 1: Same Metric Values under Different Workloads

We found that in an ideal environment, the ratio of workload to the number of instances roughly defines three phases for metrics to vary. In these three phases, latency and RPS alternately lose information, as illustrated in Fig. 3a and Fig. 3b.



(a) Distribution of latency versus workload.

(b) Distribution of RPS versus workload.



(c) The CPU usage-to-RPS ratios (CCpR) of a service across different APIs.

Figure 3: The information loss under Condition 1.

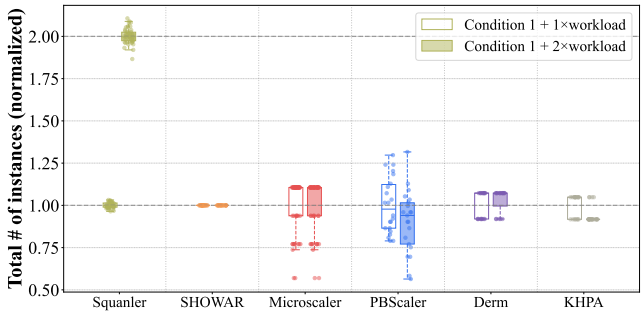


Figure 4: Allocations of autoscalers under different workloads when their selected metrics are under Condition 1.

Phase 1 is the stage where the number of instances of every service are redundant for the workload, causing requests to rarely queue. In Phase 1, latency stays constant at the average execution time of a request, resulting in a loss of information and leaving only RPS to convey information about the specific workload.

Phase 2 is the stage where the number of instances of certain services are insufficient to match the workload, leading to queue accumulation. Since the system still manages to process all requests, all return with HTTP status codes 200. In Phase 2, RPS reaches the maximum throughput of the API and ceases to increase. At this point, its information is lost. conversely, due to queuing, latency begins to grow proportionally with the workload and becomes the primary carrier of information.

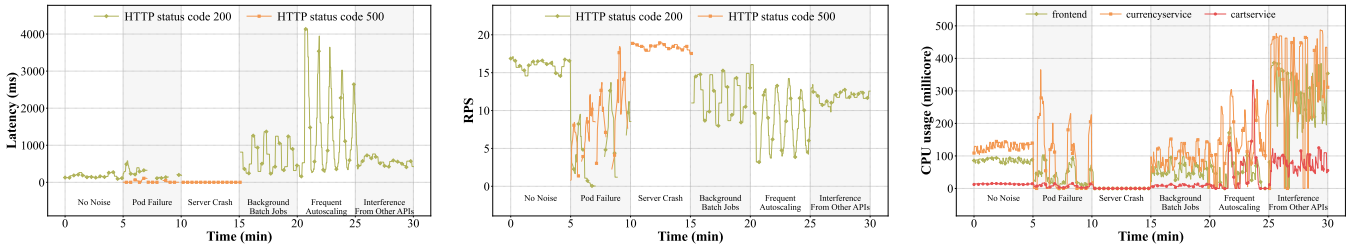
Phase 3 is the stage where the number of instances of certain services are severely insufficient to match the workload, causing the queue to overflow rather than continue accumulating. Requests either join the maximum-length queue and are processed (code 200), or they are rejected (code 500; for simplicity, we use 500 to represent all failed requests). In Phase 3, the unchanging queue length causes the latencies for both code 200 and 500 to cluster around two fixed values. Simultaneously, RPS with code 200 also remains at the maximum throughput, leaving only RPS with code 500 to increase with the workload and convey the remaining information.

The information loss of CPU usage is indirectly identified through its strong linear correlation with RPS. As a service-level metric, CPU usage cannot be directly compared with API-level workload. Therefore, we analyze the this indirectly by leveraging the relationship between CPU usage and RPS. We denote CCpR_s^i (CPU Cost per Request) as the CPU resources consumed by service s to successfully process one RPS from API i. Fig. 3c shows the CCpR samples of service s across five different APIs of the testbed application on specific hardware types (source code in Sec. 9). For each API, the CCpRs overall cluster tightly around distinct constants. Although CCpR is not constant due to various underlying mechanisms such as caching and dynamic calls, its mathematical expectation indeed exists. This implies that CPU usage and RPS (code 200) are strongly and linearly correlated. Consequently, the information loss of RPS (code 200) in Phases 2 and 3 extends to CPU usage as well.

In conclusion, Condition 1 is common and severe in latency, RPS, and CPU usage. Each of them can stay fixed under different workloads and fail to reflect it: latency stays fixed during Phase 1 and Phase 3, while RPS and CPU usage do so during Phase 2 and Phase 3. When they stay fixed, the incomplete information makes it challenging to infer the workload, thereby imposing a significant performance degradation on autoscalers that select them.

How Condition 1 undermines autoscalers. Condition 1 degrades nearly half of the actions made by autoscalers, making latency-based autoscalers ineffective at scaling in, and RPS- or CPU-based autoscalers ineffective at scaling out. Specifically, in Phase 1 where resources are redundant and need to scale in, latency-based autoscalers cannot perceive the exact workload to determine the precise scale-in amount due to the information loss of latency, and are therefore forced into a slow, trial-and-error search for the optimal allocation, leaving the system in a prolonged state of resource redundancy. Conversely, in Phase 2 where resources are insufficient and need to scale out, RPS- and CPU-based autoscalers cannot perceive the exact workload to determine the precise scale-out amount due to the information loss of RPS and CPU usage, and similarly fall into trial-and-error, leaving the system in a prolonged state of SLO violations. In Phase 3, autoscalers based on all three metrics are completely degraded.

Fig. 4 shows this inevitable degradation. SHOWAR [3] and Microscaler [34] are latency-based, PBScaler [33] and Derm [8] are RPS-based, and KHPA [10] is CPU usage-based. We maintain the system in Phase 1 when latency-based autoscalers are operating or in Phase 2 when RPS- and CPU-based autoscalers are operating, to compare their decisions under two distinct workloads when their selected metrics loses information. All baselines erroneously provide the same allocation under two distinct workloads. Only



(a) Latency is distorted by noise under a constant workload.

(b) RPS is distorted by noise under a constant workload.

(c) CPU usage is distorted by noise under a constant workload.

Figure 5: The information loss due to lack of necessity.

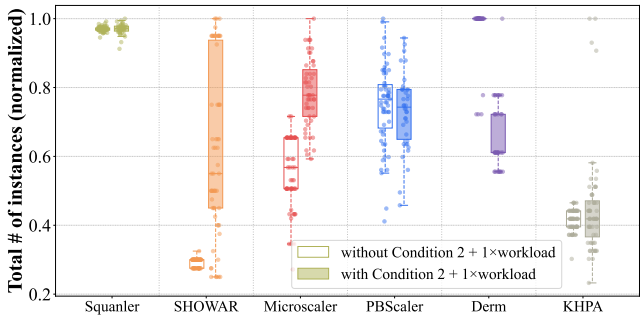


Figure 6: Allocations of autoscalers under identical workload when their selected metrics are without and with Condition 2.

the workload-based Squanler stably provides allocations that correspond to the twofold increase in workload. This is because our equation accurately quantifies the workload across all phases.

2.2 Condition 2: Different Metric Values under the Same Workloads

When metrics are simultaneously distorted by other external noise, their information also becomes incomplete. Unlike the systematic loss in the first condition, the information loss here is more stochastic.

Unpredictable noise distorts metrics, as shown in Fig. 5. Since noise is difficult to describe uniformly using numerical values, we only plot the metrics under noise. We observe that under a constant workload, metrics become inconsistent after noise are injected: we first injected pod failure (5–10 min) and server crash (10–15 min). These two types of noise cause instances to fail suddenly and stop working, making latency and RPS with code 200 fluctuate violently or even drop to zero, while triggering latency and RPS with code 500. Meanwhile, CPU usage significantly decreases as instances cannot receive requests. We then injected background batch jobs (15–30 min), frequent autoscaling (20–25 min), and invocations of other APIs within the same testbed application (25–30 min). These three types of noise consume substantial resources, leading to resource contention. Latency, RPS, and CPU usage are distorted as the intensity of contention varies; furthermore, such distortions are difficult to accurately explain or quantify.

Ideally, metrics should exclusively reflect the workload without being distorted by irrelevant noise. However, the reality is quite the opposite. Condition 2 is also common and severe in latency, RPS, and CPU usage. Pervasive noise continuously distorts the information in metrics that was already fragmented. This makes it difficult for an autoscaler to remain stable and extract useful information from these metrics.

How Condition 2 undermines autoscalers. Condition 2 affects autoscalers by making their allocations as unstable as the metrics themselves. To mitigate this—a process commonly referred to as "improving robustness"—the required countermeasures become excessively complex. For example, when instance count distorts metrics (as it can be viewed as a form of generalized noise that causes metrics to vary under the same workload), the autoscaler must collect vast amounts of data on instance count to quantify and filter out the distortion and isolate workload-related information. This introduces a curse of dimensionality, making data collection labor-intensive, limiting generalization, and requiring retraining after every update [1, 8, 24, 25, 30, 30, 33, 35]. Similarly, when pod failures and server crashes distort metrics, the autoscaler must incorporate mechanisms including but not limited to pre-cleaning and smoothing raw metrics [3, 33], setting sampling cooldown periods [31], and implementing disaster recovery logic to prevent allocations from being distorted. While these countermeasures seem reasonable, they are not inherently necessary: if metrics were entirely decoupled from instance count, pod failures, and server crashes, and other forms of noise, there would be no need to compensate for them in the first place. This is precisely how Condition 2 dictates the design complexity of autoscalers.

Without these countermeasures, the resulting allocations become distorted. As shown in Fig. 6, all baselines fail to fully overcome the distortion. Consequently, under the second test where noise is present, the resulting allocations jitter more violently compared to the first test without noise. Only the workload-based Squanler stably tracks the actual workload and suggests a constant allocation for the constant workload. Notably, Squanler does not take any countermeasures against any noise. Our equation to quantify the workload is inherently noise-free.

3 The Equation to Quantify Workload

We have redefined the workload in synchronously invoked APIs as "the user count under a specified average outside latency", and

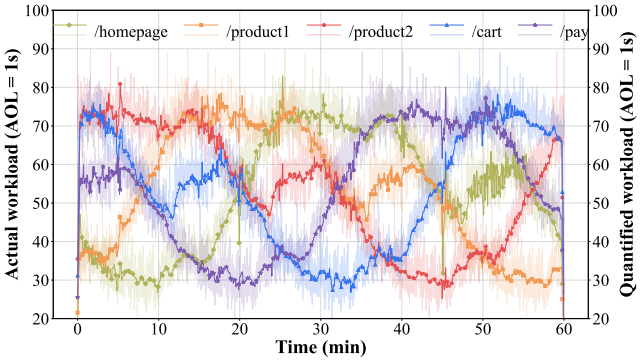


Figure 7: Eq. 2 (solid lines) accurately quantifies the actual workload that we injected (light background).

the equation to obtain it can be derived from Fig. 1. In synchronous invocations on API i with HTTP status code c, a request spends an average inside latency (AIL) within the gateway and an average outside latency (AOL) outside the gateway. This results in a total average request lifespan of AIL + AOL, during which RPS increases by 1. Thus, the average request-sending density is $1/(\text{AIL} + \text{AOL})$, the total volume is the workload, and the total mass is RPS. We relate these variables within this lifecycle analogous to the density formula:

$$\text{workload}_i^c = \text{RPS}_i^c \cdot (\text{AIL}_i^c + \text{AOL}), \tag{1}$$

where workload_i^c, RPS_i^c, and AIL_i^c represent the workload, RPS and AIL within this lifecycle. Summing the workload_i^c across all HTTP status codes for API i yields the total workload_i for API i:

$$\text{workload}_i = \sum_{c \in \text{HTTP Status Codes}} \text{RPS}_i^c \cdot (\text{AIL}_i^c + \text{AOL}). \tag{2}$$

We can manually specify any value for AOL, which determines the unit of workload_i. For example, if AOL is set to 1s and 2s separately, and the resulting workload_i = 150 (1s) or = 200 (2s), it indicates that the workload is equivalent to 150 users with an AOL of 1s, or 200 users with an AOL of 2s; both are valid quantifications. In practice, we recommend setting AOL to 2.65s. This value is derived from a classical human-computer interaction model [6] and large-scale latency measurements [19], and is highly representative of real-world production environments. Notably, workload_i must be reconstructed from AIL and RPS—the actual effects of the workload—and must incorporate an estimated AOL into the calculation. This abstract quantification distinguishes it from simple user counts—such as those collected from sessions or routers—which fail to capture the underlying system performance. This equation can quantify the actual workload of a synchronously invoked API but is not applicable to asynchronous invocation (where AIL_i^c should be omitted). Fortunately, asynchronous invocation by users is an edge case and is thus excluded from our current focus.

We then verify whether Eq. 2 can accurately quantify the actual workload as expected. This is evaluated by examining the bijective relationship between the ground-truth injected workload and the workload quantified via Eq. 2 utilizing only internal cluster information. We injected dynamic workloads into five APIs of the testbed application [18] using the oscillatory Wikipedia traffic pattern [32] as the base waveform. Compared to static workloads, dynamic workloads with significant fluctuations force the system to transition repeatedly between different phases to verify that our quantified workload is free of Condition 1. Meanwhile, we frequently injected various noise throughout the process to verify that our quantified workload is free of Condition 2. Having eliminated both conditions, we achieve a bijective quantification of the actual workload, thereby stably tracking its information.

As shown in Fig. 7, the workload quantified by Eq. 2 remains resilient to phase transitions and noise, showing no Condition 1 and 2. It closely overlaps with the ground-truth workload and effectively smooths out its drastic fluctuations. They achieve consistently high and stable correlations across all APIs, averaging 0.97 for the Pearson correlation coefficient (PCC) and 0.96 for Spearman's rank correlation coefficient (SRCC). This confirms that Eq. 2 always captures the full information of the actual workload. This result is unsurprising: observing the dynamics of latency and RPS, it is evident that within the same lifecycle, latency and RPS vary alternately with respect to workload. When one remains fixed, the other exhibits a linear correlation with the workload. Consequently, Eq. 2 integrates their complementary information, precluding the occurrence of Condition 1. Furthermore, workload act like a set of balls and HTTP status codes like a set of boxes. No matter how noise distorts the distribution of balls among the boxes, the total workload remains constant. Therefore, Eq. 2 aggregates the workload across all HTTP status codes, precluding the occurrence of Condition 2.

We have also performed this evaluation on traditional metrics (data provided in Sec. 9). The results indicate that, under the combined influence of Condition 1 and 2, the correlation between traditional metrics and the actual workload is poor. Specifically, PCC and SRCC between latency (code 200) and workload average only 0.06 and 0.05. For latency (code 500), they are 0.14 and -0.05. For RPS (code 200), they are 0.49 and 0.52, while for RPS (code 500), they are 0.05 and 0.06. Although the correlation between CPU usage and workload cannot be quantified directly, it is certain that it cannot exceed that of RPS (code 200). To illustrate, a metric with an average SRCC of 0.52 implies that an autoscaler relying on it can only capture a narrow 52% of the workload information. Even if the autoscaler is optimized to its theoretical limits, its allocation decisions will, on average, only achieve a 0.52 match with the actual workload. This information incomplete creates a fundamental performance barrier between different types of autoscalers, allowing us to anticipate the performance of different autoscalers even before experimental comparison.

Note that we are not pursuing a perfect numerical match between the ground-truth workload and the workload quantified by Eq. 2, but rather a bijectivity between them. Although their absolute values will differ when the actual configured AOL and our assumed AOL are not identical, the bijectivity remains invariant. It is the complete information that matters most.

4 Squanler: Workload-Based Microservice Autoscaler

We develop Squanler to demonstrate the significant advantages of using our quantified workload over traditional metrics. However, it should be noted that Squanler is merely a preliminary demonstration; it does not imply that our quantified workload is limited to this specific implementation, as it can also be integrated into existing autoscalers to enhance their workload awareness.

4.1 Key Design

The core idea of Squanler is to reposition the average inside latency (AIL) and RPS of each API toward a target equilibrium by leveraging the dynamic interplay between them.

1) Squanler quantifies the current workload$_i$ of API i via Eq. 2.

2) Squanler derives the target throughput (denoted as RPS_i^*) that would theoretically occur if the current workload were all concentrated into a successful lifecycle with AIL = SLO, where SLO is our desired latency threshold:

$$\begin{aligned}\text{RPS}_i^* &= \frac{\text{workload}_i}{\text{SLO} + \text{AOL}} \\ &= \frac{\sum_{c \in \text{HTTP Status Codes}} \text{RPS}_i^c \cdot (\text{AIL}_i^c + \text{AOL})}{\text{SLO} + \text{AOL}},\end{aligned} \tag{3}$$

where RPS_i^c and AIL_i^c can be queried from Prometheus [11] by specifying the API i, HTTP code c, and service *gateway*, while AOL and SLO are set manually.

3) Squanler leverages the strong linear relationship between CPU usage and RPS to calculate the precise CPU millicores required for each service to regulate RPS: given the average CCpR_i^s (CPU Cost per Request) of service s relative to API i, granting API i a throughput of RPS_i^* implies that service s will consume approximately $\text{RPS}_i^* \cdot \text{CCpR}_i^s$ millicores of CPU, and allocating this specific amount of CPU to service s can conversely regulate its throughput to RPS_i^*.

4) Squanler integrates all I APIs and S services: let $\text{RPS}_{1\times I}^*$ denote the vector composed of RPS_i^* for I APIs and $\text{CCpR}_{I\times S}$ denote the matrix of CCpRs for S services across I APIs. Then, $\text{Allocation}_{1\times S}$, the required CPU millicores for each service to exactly match the current workload of I APIs, is:

$$\text{Allocation}_{1\times S} = \text{RPS}_{1\times I}^* \cdot \text{CCpR}_{I\times S}. \tag{4}$$

Applying $\text{Allocation}_{1\times S}$ to the S services allows us to maintain latencies near the SLO. Sec. 5.3.1 verifies that it is indeed the required resources under ideal environments.

4.2 Secondary Detail

A specific case requires additional preprocessing: some APIs exhibit causal dependencies, whereby requests on a parent API automatically trigger subsequent requests on a child API. For example, in Online Boutique [18], `/setCurrency` returns code 302 to redirect users to `/homepage`. As a result, every requests through `/setCurrency` generate attached requests to `/homepage`. From a user's perspective, we only inject one workload on `/setCurrency`. But from a system perspective, separate workloads are observed on both `/setCurrency` and `/homepage`. While the latter is the actual output of Eq. (2), the former is the intended output we seek. This is because the latter introduces slight discrepancies into the derivation of $\text{Allocation}_{1\times S}$: when collecting CCpR, we inject workload into API i in isolation and define CCpR_i^s as the mean CPU usage per unit of RPS for service s on a specific hardware model. When `/setCurrency` is attached to `/homepage`, the measured $\text{CCpR}^s_{\texttt{/setCurrency}}$ actually already reflects the combined resource unit triggered by both APIs. Therefore, we want our quantified workload to be combined, too.

The solution is to perform the following preprocessing on the metrics before applying Eq. (2). For a redirect chain such as:

$$\texttt{/A} \rightarrow \texttt{/B} \rightarrow \ldots \rightarrow \texttt{/Y} \rightarrow \texttt{/Z}, \tag{5}$$

Eq. (6) and Eq. (7) should be applied to preprocess AIL and RPS of each API, merging workload fragments. The APIs should be traversed in the left-to-right order defined by the index sets:

$$\text{AIL}_i^{302} \leftarrow \text{AIL}_i^{302} + \text{AIL}^c_{\text{direct child of } i}, \quad i \in \{\texttt{/Y}, \ldots, \texttt{/A}\}, \tag{6}$$

$$\text{RPS}_i^c \leftarrow \text{RPS}_i^c - \text{RPS}^{302}_{\text{direct parent of } i}, \quad i \in \{\texttt{/Z}, \ldots, \texttt{/B}\}, \tag{7}$$

where $c = 200$ (indicating a successful redirect chain) or 500 (representing an unsuccessful, broken redirect chain) if the corresponding API is `/Z`, otherwise $c = 302$. This solution allows the workload fragmented by the redirect chain to be consolidated back into the total workload of `/A`, the only API invoked by users.

5 Evaluation

This section compares Squanler with several state-of-the-art autoscalers.

5.1 Evaluation Environment

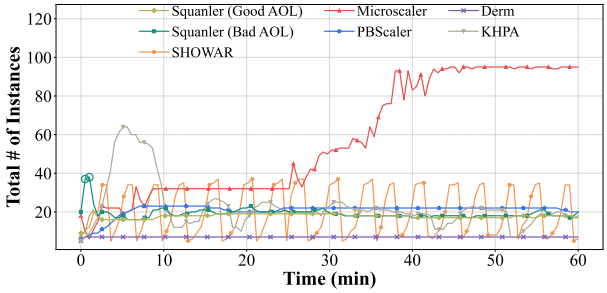
Environment Setup. We deployed two open-source microservice applications, Sock Shop [27] and Online Boutique [18], on a local cluster with one master node (16 CPU cores, 32 GB RAM) and two worker nodes (totaling 40 CPU cores, 128 GB RAM). The cluster used Docker [12] v27.5.1 and Kubernetes [9] v1.20.4. The AOL configured during workload injection is 1s, and the SLO of P90 latency is 500ms.

Baselines. We compared Squanler with five autoscalers: two latency-based, two RPS-based, and one CPU usage-based.

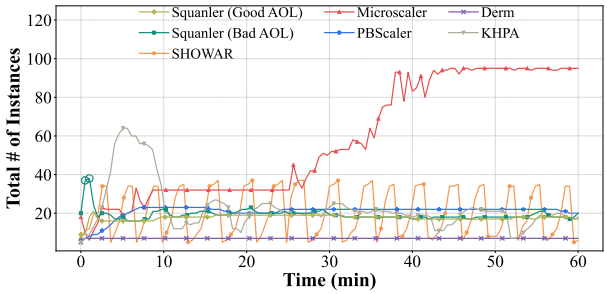
- **SHOWAR [3].** This latency-based autoscaler maintains the latency of each service via PID balancing equations.
- **Microscaler [34].** This latency-based autoscaler optimizes the trade-off between instance count and latency via Bayesian iteration.
- **PBScaler [33].** This RPS-based autoscaler identifies bottleneck services, then optimizes their resource allocation through genetic algorithms.
- **Derm [8].** This RPS-based autoscaler models each service's latency and accounts for dynamic graphs, heuristically minimizing resource allocation and SLO violations.
- **KHPA [10].** This CPU usage-based autoscaler maintains the CPU utilization of each service at a target percentage by dynamically scaling the number of instances up or down.

We tested two variants of Squanler:

- **Squanler (Good AOL).** It sets a "Good" AOL = 1s that is closely aligned with the configured AOL in workload files, representing the best-case performance of Squanler.

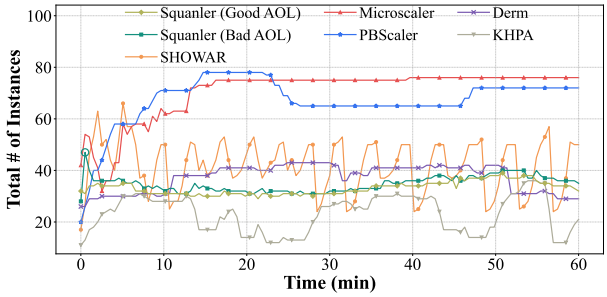


(a) Allocations (total number of instances) across autoscalers.

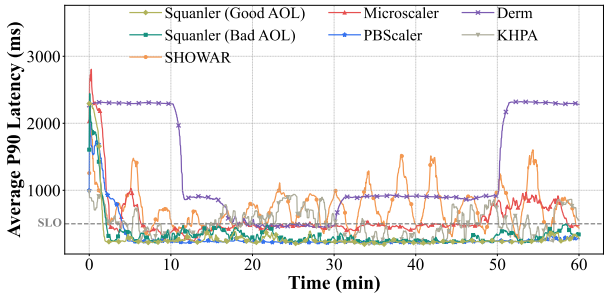


(b) Average P90 latency across autoscalers.

Figure 8: Comparison across autoscalers on Sock Shop under dynamic workloads.



(a) Allocations (total number of instances) across autoscalers.



(b) Average P90 latency across autoscalers.

Figure 9: Comparison across autoscalers on Online Boutique under dynamic workloads.

- **Squanler (Bad AOL).** It sets a "Bad" AOL = 0.2s that is far from the configured AOL. Note the rapid convergence of Eq. (3), where $\Delta\text{RPS}^*_{0.2\to1} \gg \Delta\text{RPS}^*_{1\to500}$. Thus, this AOL was selected to expose the worst-case performance of Squanler.

5.2 Comparison Under Dynamic Workloads

We choose the Wikipedia workload from March 16, 2015 [32] as the workload waveform for the two benchmarks, to test the ability of each autoscaler to maintain the SLO and minimize resource redundancy.

Overall (Fig. 8 and Fig. 9), Squanler outperforms all other autoscalers. From a metric perspective, this is because it consistently obtains nearly 100% of workload information. This allows Squanler to bypass the detrimental trial-and-error process and match appropriate resources to the current workload in almost a single step. In contrast, other baselines only obtain fragmented workload information; even when such information is fully utilized, they struggle to overcome the performance barriers introduced by their chosen metrics.

Fig. 8a and Fig. 9a show the total allocation across autoscalers. Without violating SLOs, both variants of Squanler allocate the lowest average instance counts on both Sock Shop (17.66 and 19.24) and Online Boutique (33.14 and 34.72). Due to the impact of AOL mismatch, Squanler (Bad AOL) initially overallocates but quickly aligns with Squanler (Good AOL), as highlighted by the circles in the figure. This is because the rapidly decreasing latency enhances the convergence of Eq. (3), thereby significantly mitigating the impact of AOL mismatch. Fig. 8b and Fig. 9b show the average P90 latency across autoscalers. Both variants of Squanler achieve the lowest SLO violation rates on both Sock Shop (5.27% and 6.76%) and Online Boutique (2.81% and 3.46%), along with the fastest reduction speed in latency, which can be attributed to their ability to produce high-quality allocations in almost a single step.

For other baselines, the incomplete information of their selected metrics imposes an ceiling on their performance, while their individual designs determine their specific position relative to that ceiling. Since the performance degradation caused by metrics has been confirmed in Sec. 2, we primarily analyze their performance from design perspectives.

For SHOWAR and Microscaler, mapping optimal allocation through latency is inherently unsuitable. Their performance is further degraded by an inherent trial-and-error style and a heavy reliance on empirical hyperparameters: SHOWAR uses latency as the input for its PID equations, increasing or decreasing allocation by a fixed β percentage each time. This design inherently leads to trial-and-error. Its performance heavily depends on PID parameters and various manual thresholds. When these are set poorly, it exhibits the typical oscillations seen in the figure. The PID equation balances a system through frequent micro-adjustments, which here means constant starting and killing of instances. However, since instance startup and destruction take far too long, this trial-and-error approach leads to prolonged SLO violations in such a slow-responding system. Instead, what is urgently needed is the ability to provide a near-optimal allocation in a single step. Consequently, SHOWAR oscillates severely, aggressively scaling down when the SLO is barely

met and abruptly scaling up when the SLO is violated, performing poorly in both SLO maintenance and resource savings.

Microscaler starts a search once the ratio of P50 to P90 latency triggers a certain threshold, and it uses Bayesian optimization within a range of [0, maximum allowable allocation] to find an instance count that minimizes its cost function. Its performance also depends heavily on hyperparameters: when the search interval is set too wide, Microscaler starts from an excessive allocation and slowly trials through errors—a style as inefficient as SHOWAR's. This is why Microscaler consistently maintains the highest allocation. The resource contention caused by maintaining so many redundant instances even leads to an eventual increase in latency.

For PBScaler and Derm, mapping required resources solely through RPS is also unsuitable. Their performance is further degraded by labor-intensive data collection and limited generalizability: the essence of PBScaler is to retrieve feasible allocations from training data based on the current RPS. Therefore, it relies heavily on the quality of the training data. In Sock Shop, where there are few critical services and a small sample space, which makes high-quality training data easy to collect and enables PBScaler to perform well. However, in Online Boutique, the addition of even a few critical services causes the sample space to become massive, making it difficult to collect high-quality training data. As a result, PBScaler can only rely on low-quality allocations when the encountered workload has not been properly explored.

Derm pre-collects traces to fit latency-RPS profiles for each service, then calculates allocations based on these profiles and current RPS to meet the SLO. However, modeling latency as a function of RPS is inherently invalid under synchronous calls. As discussed, latency and RPS are mutually exclusive metrics that covary around workload in synchronous calls. They are orthogonal, same-level metrics without a causal relationship. This fundamental flaw likely misled Derm into allocating resources to the wrong services in Online Boutique, resulting in its near-failure to maintain the SLO.

In addition, Condition 1 of RPS directly leads to the poor scaling out of PBScaler and Derm. PBScaler takes over management when the system is in Phase 2. It clearly falls into trial-and-error during scale-out, with its allocation climbing slowly rather than converging in a single step, which prevents it from rapidly reducing latency. Unlike PBScaler, Derm fails to overcome the information loss of RPS. It is misled by the low RPS caused by the limited maximum throughput of current API, mistakenly assuming the actual workload is also low. Consequently, it consistently provides insufficient allocations and frequently fails to maintain the SLO.

KHPA directly adjusts the number of instances for each service to ⌈current CPU usage / target CPU usage⌉. While its design is simple and robust, its poor performance stems primarily from the inherent limitations of CPU usage as a metric. In Sock Shop, KHPA aggressively and continuously allocated excessive instances to `shippingservice`, a mock service. This occurred because the CPU overhead incurred during instance startup misled KHPA, triggering a chain reaction that scaled `shippingservice` up to the maximum allowable allocation. In Online Boutique, KHPA exhibited a slow climb during the initial stages. This likely resulted from a trial-and-error process occurring because the workload was relatively high, which led to CPU usage losing information in Phase 2. Furthermore, the CPU metrics provided by cAdvisor were unstable, often returning null or stale data. This forced KHPA into erroneous scale-in actions, a phenomenon that consistently appeared across multiple experimental runs.

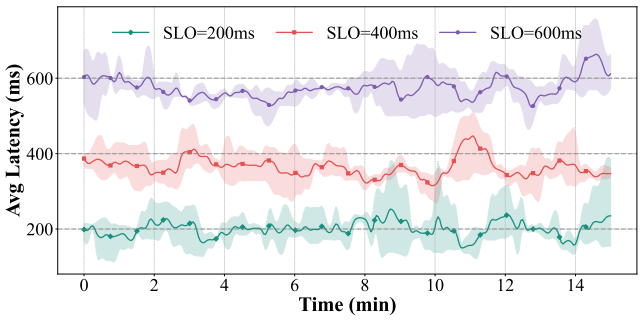


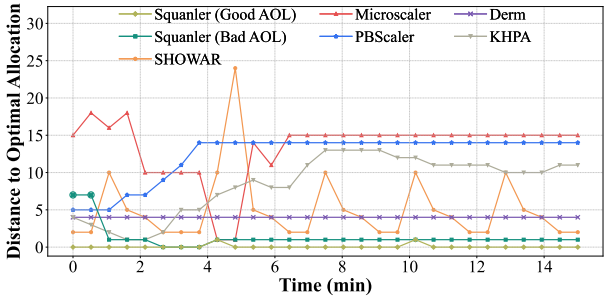
Figure 10: Squanler's near-optimal allocation maintains the average latency consistently close to the SLOs.

5.3 Why does Squanler Outperform Other Baselines

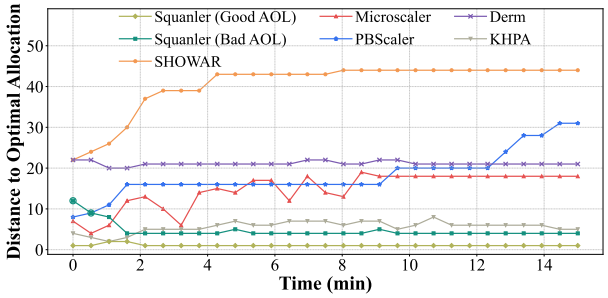
We can also explain why Squanler outperforms others by analyzing its own unique strengths. In an ideal environment, a near-optimal autoscaler that is difficult to be significantly outperformed is one that: 1) provides a near-optimal allocation that maintains latency strictly at the SLO; 2) provides this allocation in almost a single step without trial-and-error; 3) operates stably under noise. This section demonstrates that Squanler nearly achieves all of these objectives.

5.3.1 Squanler achieves near-optimal allocation. The ability to calculate near-optimal allocations with millicore-level precision is the first reason why Squanler outperforms others. According to our theory, in an ideal environment with a "Good AOL" profile, Squanler can derive the required resources that maintains the average latency precisely at the SLO. To verify this, we deployed Squanler (Good AOL) to control an Online Boutique API. We tested three SLOs—200ms, 400ms, and 600ms—to observe whether latency would stabilize near them as expected. As shown in Fig. 10, despite inevitable noise-induced fluctuations, Squanler (Good AOL) consistently maintains the average latency near the SLOs, demonstrating that Squanler's allocation is indeed near-optimal. To our knowledge, most existing autoscalers still operate at the coarse-grained level of per-service instance counts, rather than the fine-grained level of per-service CPU millicores. Such discrete allocation is incapable of the high-precision task of controlling latency to a continuous SLO.

5.3.2 Squanler converges to near-optimal in almost a single step. The ability to achieve near-optimal allocation in almost a single step without trial-and-error is the second reason why Squanler outperforms others. To verify this, we randomly tested six static workloads across two benchmarks. For each workload, we first manually determined the corresponding ground-truth optimal allocation, then compared the L1 distance between the optimal allocation and each autoscaler's allocation, to measure how many steps each autoscaler took to converge to the optimal allocation. Fig. 11a illustrates the two most representative cases, while the remaining data can be found in Sec. 9. In every test case, Squanler (Good AOL) converges

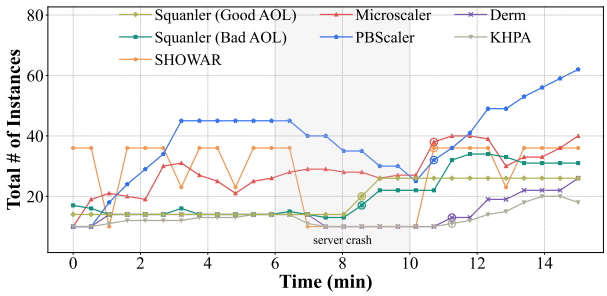


(a) L1 distance across autoscalers on Sock Shop.

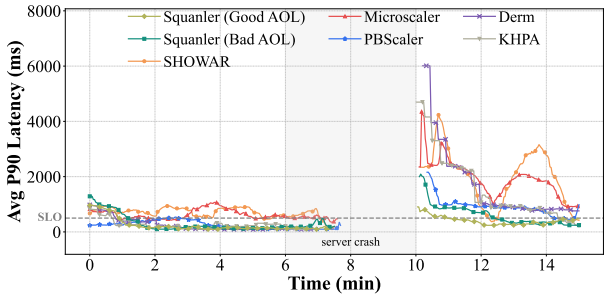


(b) L1 distance across autoscalers on Online Boutique.

Figure 11: Squanler (Good AOL) finds the near-optimal allocation in a single step. Squanler (Bad AOL) and Squanler (Good AOL) are approximate when the latency is brought under control.



(a) Allocation across autoscalers under robustness tests. Circles denote when each autoscaler detects the workload increase.



(b) Average P90 latency across autoscalers under robustness tests.

Figure 12: Squanler can maintain stable operation under severe noise interference.

to the near-optimal allocation in a single step. It exhibits the smallest distance to the optimum and achieves remarkable stability, with an average L1 distance of only 0.59 across all tests. When initially managing a high-latency API, Squanler (Bad AOL) provides a temporary redundant allocation (indicated by the circles in the figure). However, as the latency is brought under control, it quickly converges and performs comparably to Squanler (Good AOL), with an average L1 distance of 3.16 across all tests. In contrast, other baselines struggle to converge to near-optimality even when provided with sufficient time and static workloads, for the metrics they rely on are inherently information-deficient, and they fail to fully exploit the available information as well.

5.3.3 Squanler maintains stability under severe noise interference. The ability to operate stably under severe noise interference is the third reason why Squanler outperforms others. Its exceptional robustness has already been verified in Sec. 2.2. Therefore, this section focuses on the specific advantages that this robustness brings to Squanler. Specifically, we injected a two-stage workload into Online Boutique: the workload is small before minute 8, but spikes abruptly and stays high after minute 8, meaning the system needs to scale out immediately. However, from minute 6 to minute 10, we injected a four-minute server crash on the servers hosting the frontend service. This severe noise prevents external traffic from being forwarded to the internal instances. creating a "blackout window" where metrics fluctuate and drop significantly, thereby misleading other baselines into erroneous scale-in (Fig. 12a) because they are not robust enough. Only Squanler robustly tracks the actual workload during the server crash and immediately recommends scaling out after detecting the workload increases at minute 8. Thus, it is well-prepared for the traffic surge when the server recovers, maintaining a lower latency (Fig. 12b). This foresight stems from the fact that Squanler only requires metrics collected at the gateway (which remains operational) to stably quantify the workload of each API within Online Boutique. Consequently, metric distortions caused by internal noise within Online Boutique fail to interfere Squanler. In contrast, all baselines are misled into incorrectly scaling in during the server crash and only begin to scale out after the server has recovered. This delay leads to significantly higher latency during the traffic surge.

Collectively, these three reasons make Squanler a near-optimal autoscaler that is hard to outperform in ideal environments, explaining why it consistently exceeds the baselines.

6 Discussion

6.1 Internal Validity

Environmental noise and randomness. A primary threat to internal validity is the reproducibility and stability of evaluation results. To address this, we eliminated all possible noise in the cluster and injected the workload within a local area network to minimize uncontrollable network latency. To prevent accidental performance fluctuations in the baselines, we ran each experiment several times, and the results showed negligible differences across these runs.

Hyper-parameter sensitivity. Another threat is the impact of the hyper-parameter AOL. For Squanler, different AOL values result in different allocations. This implies that the resources required to balance the workload, when quantified with different units, will differ slightly. This is theoretically unavoidable, but entirely acceptable because the actual difference is very small: consider an extreme edge case where an API has an AIL = 2000ms and an SLO = 200ms. We define $AOL_1 = 10^9$s and $AOL_2 = 2.65$s $\ll AOL_1$. We find that due to the rapid convergence of Eq. 3, the $\text{Allocation}_{1\times S}$ under AOL_2 is only 0.63× larger than that under AOL_1. Furthermore, this gap narrows even further as the 0.63× redundant allocation triggers a rapid decline in AIL. This limited impact is also confirmed in Sec. 5: evaluation shows that despite a large AOL difference, the average resource allocation of Squanler (Bad AOL) was only about 9.37% higher than Squanler (Good AOL). This results in only a short-period, minor overallocation.

6.2 External Validity

Representativeness of observed dynamics in traditional metrics. The first threat to external validity is whether the dynamics of latency, RPS, and CPU usage observed in the testbed application are representative. For latency and RPS, their dynamics arise from the queueing of requests. Essentially, regardless of the diversity in internal API implementations, requests will naturally form a queue when waiting to be processed. Therefore, the dynamics observed in the testbed application are equally applicable to any synchronously invoked API. For CPU usage, although business logic may differ across services, they are ultimately compiled into machine code, and the execution of machine code on the CPU is fundamentally similar. Thus, the observations from the testbed application are representative along the CPU dimension as well. The great performance of Squanler in experiments also validates this point.

Choice of benchmark applications. The second threat is whether the benchmark applications we selected are sufficiently representative. We did not choose several larger-scale benchmarks because we found that they were not well suited to autoscaling studies for the following reasons: 1) for applications like Train Ticket [36], written in Java, have excessively slow startup times. This introduces a significant lag in the execution of scaling decisions, meaning that by the time an autoscaler's action takes effect, the workload may have already shifted. Such misalignment leads to degraded performance across all autoscalers, making it difficult to distinguish their relative efficacy. 2) for applications like Social Network [15], some of their performance bottlenecks lie in the stateful database, which is not suitable for direct scaling, as this might compromise data consistency and lead to errors. 3) Another common issue shared by applications like Social Network and Media Microservices [15] is that nearly half of their services are databases, meaning that the number of scalable business services is in fact not significantly higher than those in Sock Shop [27] and Online Boutique [18] benchmarks we adopted.

7 Related Work

We classify and summarize current autoscalers by the primary metrics they adopt.

Latency-based autoscalers. Microscaler [34] quantifies the service power using P90 and P50 latency, with decisions derived through Bayesian optimization. ATOM [16] optimizes resource allocation using latency, RPS, and CPU usage within the LQN model. GRAF [24] uses MPNN to predict the possible latency of a candidate allocation strategy. SHOWAR [3] builds PID equations on latency. Sinan [35] uses a hybrid CNN-Boosting Tree model to predict short-term latency and long-term SLO violation. Derm [8] models latency with exponential distributions. MSARS [21] integrates RPS-latency profiles as a partial input to its RL model.

RPS-based autoscalers. Chameleon [5] uses sARIMA and tBATS to forecast future workloads based on RPS. Burst-Aware [1] handles sudden load spikes by identifying surges in RPS. Erms [23] formulates a convex optimization problem with current RPS as input, deriving resource-minimizing allocations under SLO constraints. DRPC [4] predicts future RPS via LSTM. DeepScaling [30] uses STGNN for RPS prediction. Humas [22] detects changes in application patterns using RPS and CPU usage. StatuScale [31] detects RPS patterns and switches strategies between two load types.

Resource usage-based autoscalers. Autopilot [26] leverages historical resource usage data and machine learning models to optimize resource allocation. KHPA [10] maintains resource utilization near predefined thresholds. MSARS [21] utilizes CPU usage as a state feature in its reinforcement learning loop to guide adaptive scaling decisions. PBScaler [33] makes scale-down decisions by analyzing historical resource usage fluctuations.

These autoscalers ultimately rely on these metrics. Although their designs are often sophisticated, their performance remains potentially undermined by the incomplete information of the metrics they rely on.

8 Conclusions

To the best of our knowledge, this paper is the first to identify the inherent informational incompleteness of latency, RPS, and CPU usage when applied to autoscaling of synchronously invoked APIs in microservices systems. We propose a more appropriate method to quantify the workload of such APIs with high precision and robustness. Based on this, we developed Squanler, a workload-based autoscaler that allocates the precise resources needed for the current workload in nearly a single step. Comparative experiments verify that Squanler consistently outperforms existing state-of-the-art autoscalers by maintaining the lowest SLO violation rates and minimal resource redundancy. In the future, we plan to extend this quantification method to a broader range of microservice governance tasks beyond autoscaling.

9 Data Availability Statement

All data and codes have been publicized [20].

References

[1] Muhammad Abdullah, Waheed Iqbal, Josep Lluis Berral, Jorda Polo, and David Carrera. 2020. Burst-aware predictive autoscaling for containerized microservices. *IEEE Transactions on Services Computing* 15, 3 (2020), 1448–1460.

[2] Apache Software Foundation. 2024. Apache JMeter: Open source load testing tool. https://jmeter.apache.org/.

[3] Ataollah Fatahi Baarzi and George Kesidis. 2021. Showar: Right-sizing and efficient scheduling of microservices. In *Proceedings of the ACM Symposium on Cloud Computing*. 427–441.

[4] Haoyu Bai, Minxian Xu, Kejiang Ye, Rajkumar Buyya, and Chengzhong Xu. 2024. Drpc: Distributed reinforcement learning approach for scalable resource provisioning in container-based clusters. *IEEE Transactions on Services Computing* (2024).

[5] André Bauer, Nikolas Herbst, Simon Spinner, Ahmed Ali-Eldin, and Samuel Kounev. 2018. Chameleon: A hybrid, proactive auto-scaling mechanism on a level-playing field. *IEEE Transactions on Parallel and Distributed Systems* 30, 4 (2018), 800–813.

[6] Stuart K Card, Thomas P Moran, and Allen Newell. 1980. The keystroke-level model for user performance time with interactive systems. *Commun. ACM* 23, 7 (1980), 396–410.

[7] Lianping Chen. 2018. Microservices: architecting for continuous delivery and DevOps. In *2018 IEEE International conference on software architecture (ICSA)*. IEEE, 39–397.

[8] Liao Chen, Shutian Luo, Chenyu Lin, Zizhao Mo, Huanle Xu, Kejiang Ye, and Chengzhong Xu. 2024. Derm: SLA-aware Resource Management for Highly Dynamic Microservices. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 424–436.

[9] Cloud Native Computing Foundation. 2021. Kubernetes: Container orchestration platform. https://kubernetes.io/.

[10] Cloud Native Computing Foundation. 2025. Horizontal Pod Autoscaler. https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/.

[11] Cloud Native Computing Foundation. 2025. Prometheus: Open-source systems monitoring and alerting toolkit. https://prometheus.io/.

[12] Docker, Inc. 2025. Docker: Containerization platform. https://www.docker.com/.

[13] Nicola Dragoni, Saverio Giallorenzo, Alberto Lluch Lafuente, Manuel Mazzara, Fabrizio Montesi, Ruslan Mustafin, and Larisa Safina. 2017. Microservices: yesterday, today, and tomorrow. *Present and ulterior software engineering* (2017), 195–216.

[14] Jonas Fritzsch, Justus Bogner, Alfred Zimmermann, and Stefan Wagner. 2018. From monolith to microservices: A classification of refactoring approaches. In *International Workshop on Software Engineering Aspects of Continuous Development and New Paradigms of Software Production and Deployment*. Springer, 128–141.

[15] Yu Gan, Yanqi Zhang, Dailun Cheng, Ankitha Shetty, Priyal Rathi, Nayan Katarki, Ariana Bruno, Justin Hu, Brian Ritchken, Brendon Jackson, et al. 2019. An open-source benchmark suite for microservices and their hardware-software implications for cloud & edge systems. In *Proceedings of the twenty-fourth international conference on architectural support for programming languages and operating systems*. 3–18.

[16] Alim Ul Gias, Giuliano Casale, and Murray Woodside. 2019. ATOM: Model-driven autoscaling for microservices. In *2019 IEEE 39th International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 1994–2004.

[17] Will Glozer. 2024. wrk: Modern HTTP benchmarking tool. https://github.com/wg/wrk.

[18] Google Cloud Platform. 2020. Online Boutique: Cloud-Native Microservices Demo. https://github.com/GoogleCloudPlatform/microservices-demo.

[19] Chuanxiong Guo, Lihua Yuan, Dong Xiang, Yingnong Dang, Ray Huang, Dave Maltz, Zhaoyi Liu, Vin Wang, Bin Pang, Hua Chen, et al. 2015. Pingmesh: A large-scale system for data center network latency measurement and analysis. In *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication*. 139–152.

[20] Holistic Workload Quantification in Synchronous APIs for Near-One-Step Microservice Autoscaling. 2026. https://anonymous.4open.science/r/Data-89D5.

[21] Kan Hu, Linfeng Wen, Minxian Xu, and Kejiang Ye. 2024. MSARS: A Meta-Learning and Reinforcement Learning Framework for SLO Resource Allocation and Adaptive Scaling for Microservices. *arXiv preprint arXiv:2409.14953* (2024).

[22] Qin Hua, Dingyu Yang, Shiyou Qian, Jian Cao, Guangtao Xue, and Minglu Li. 2024. Humas: A heterogeneity-and upgrade-aware microservice auto-scaling framework in large-scale data centers. *IEEE Trans. Comput.* (2024).

[23] Shutian Luo, Huanle Xu, Kejiang Ye, Guoyao Xu, Liping Zhang, Jian He, Guodong Yang, and Chengzhong Xu. 2022. Erms: Efficient resource management for shared microservices with sla guarantees. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. 62–77.

[24] Jinwoo Park, Byungkwon Choi, Chunghan Lee, and Dongsu Han. 2021. GRAF: A graph neural network based proactive resource allocation framework for SLO-oriented microservices. In *Proceedings of the 17th International Conference on emerging Networking EXperiments and Technologies*. 154–167.

[25] Haoran Qiu, Subho S Banerjee, Saurabh Jha, Zbigniew T Kalbarczyk, and Ravishankar K Iyer. 2020. {FIRM}: An intelligent fine-grained resource management framework for {SLO-Oriented} microservices. In *14th USENIX symposium on operating systems design and implementation (OSDI 20)*. 805–825.

[26] Krzysztof Rzadca, Pawel Findeisen, Jacek Swiderski, Przemyslaw Zych, Przemyslaw Broniek, Jarek Kusmierek, Pawel Nowak, Beata Strack, Piotr Witusowski, Steven Hand, et al. 2020. Autopilot: workload autoscaling at google. In *Proceedings of the Fifteenth European Conference on Computer Systems*. 1–16.

[27] Sock-Shop Contributors. 2016. Sock-Shop: A Microservices Demo Application. https://github.com/microservices-demo/microservices-demo.

[28] Jacopo Soldani, Damian Andrew Tamburri, and Willem-Jan Van Den Heuvel. 2018. The pains and gains of microservices: A systematic grey literature review. *Journal of Systems and Software* 146 (2018), 215–232.

[29] The Locust Developers. 2021. Locust: Open source load testing tool. https://locust.io/.

[30] Ziliang Wang, Shiyi Zhu, Jianguo Li, Wei Jiang, KK Ramakrishnan, Meng Yan, Xiaohong Zhang, and Alex X Liu. 2024. Deepscaling: Autoscaling microservices with stable cpu utilization for large scale production cloud systems. *IEEE/ACM Transactions on Networking* (2024).

[31] Linfeng Wen, Minxian Xu, Sukhpal Singh Gill, Muhammad Hilman, Satish Narayana Srirama, Kejiang Ye, and Chengzhong Xu. 2025. StatuScale: Status-aware and elastic scaling strategy for microservice applications. *ACM Transactions on Autonomous and Adaptive Systems* 20, 1 (2025), 1–25.

[32] Wikimedia Foundation. 2015. Wikimedia Pageviews Dataset. https://dumps.wikimedia.org/other/pageviews/.

[33] Shuaiyu Xie, Jian Wang, Bing Li, Zekun Zhang, Duantengchuan Li, and Patrick CK Hung. 2024. PBScaler: A bottleneck-aware autoscaling framework for microservice-based applications. *IEEE Transactions on Services Computing* (2024).

[34] Guangba Yu, Pengfei Chen, and Zibin Zheng. 2019. Microscaler: Automatic scaling for microservices with an online learning approach. In *2019 IEEE International Conference on Web Services (ICWS)*. IEEE, 68–75.

[35] Yanqi Zhang, Weizhe Hua, Zhuangzhuang Zhou, G Edward Suh, and Christina Delimitrou. 2021. Sinan: ML-based and QoS-aware resource management for cloud microservices. In *Proceedings of the 26th ACM international conference on architectural support for programming languages and operating systems*. 167–181.

[36] Xiang Zhou, Xin Peng, Tao Xie, Jun Sun, Chenjie Xu, Chao Ji, and Wenyun Zhao. 2018. Benchmarking microservice systems for software engineering research. In *Proceedings of the 40th International Conference on Software Engineering: Companion Proceeedings*. 323–324.

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009